

# Timeline & Current Update

Share with us what the major skills you're challenging and what your personal timeline & goals are. Share what problem you are currently working on in your project, this should show *progress* since your last update. If you are stalled, share why and any resources or reference material you are working through.

## 9 Project Timeline

| Window               | Milestone / Work Product                                                                                         | Output                                |
|----------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Late Jan – Early Feb | Finalize proposal and baseline parameters ( $S$ , $K$ , $r$ , $T$ , $\sigma$ ) and clean notebook structure      | Proposal PDF + cleaned Colab notebook |
| Feb 10 – Feb 23      | Implement full CRR workflow (tree construction, payoff computation, backward induction), add visualization tools | Stock tree + option tree visuals      |
| Feb 24               | Mini Lecture (Practical): Binomial Tree (CRR)                                                                    | 1-page cheat sheet + demo notebook    |
| Feb 25 – Mar 16      | Run experiments: convergence analysis ( $N$ sweep), runtime scaling, volatility sensitivity                      | Figures + results tables              |
| Mar 17               | Mini Lecture (Theory): No-arbitrage and risk-neutral valuation                                                   | Resource page + activity prompts      |
| Mar 18 – Early Apr   | Write paper draft (methods, results, discussion) and refine visualizations                                       | Draft academic report                 |
| Early – Mid Apr      | Final revisions, literature review polishing, GitHub packaging and reproducibility checklist                     | Final paper + GitHub repository       |
| Apr 22               | Final presentation (SPARK Symposium)                                                                             | Slides/poster + presentation          |

Table 1: Planned project timeline and deliverables.

### Structure of Code

Data\_loader.py → stock price  
volatility.py → sigma estimate  
Fred\_loader.py → risk-free rate  
Binomial\_model.py → CRR pricing  
Black\_scholes.py → BS pricing

experiments.py → convergence experiments  
Tree\_visualization.py → plots

### **run\_experiments.py (Main driver)**

Imports all project modules from src/.

Executes the pipeline:

- pulls market inputs (spot price + risk-free rate)
- estimates volatility
- builds binomial trees + visualizes
- runs convergence experiments (varying N)
- compares vs Black–Scholes and plots results

Output: printed results + saved/displayed figures.

### **src/data\_loader.py (Market Data)**

Pulling spot price  $S_0$  & Historical price series

yfinance:

- Download most recent daily data point
  - `data = yf.Ticker(ticker).history(period="1d")`
- Extract latest closing price
  - `spot_price = float(data["Close"])`

Output: current price and prices.

### **src/volatility.py (Market Data)**

Computes historical volatility from price data

- Converts prices to log returns
  - `data = yf.Ticker(ticker).history(period=lookback)` - Historical price data
- Computes standard deviation for returns
  - `spot_price = float(data["Close"])`
- Annualizes using  $\sqrt{252}$  for trading days

Output: a single volatility estimate (float) used by CRR and Black-Scholes

### **src/fred\_loader.py (Risk-free rate retrieval)**

Estimate annualized historical volatility from market data. → undocumented but official shortcut to download any series as a CSV without an API key (Federal Reserve Economic Data)

- Fetches a risk-free proxy (e.g., 3-month or 10-year Treasury yield) from FRED.
- Returns it as a decimal annual rate (e.g., 0.05 instead of 5).
- This plugs into:
  - Risk-neutral probability in CRR
  - Discount factor in Black-Scholes

Output: risk-free rate float.

| A          | B      |
|------------|--------|
| observatio | DGS3MO |
| 9/1/1981   | 17.01  |
| 9/2/1981   | 16.65  |
| 9/3/1981   | 16.96  |
| 9/4/1981   | 16.64  |
| 9/7/1981   |        |
| 9/8/1981   | 16.54  |
| 9/9/1981   | 16.39  |
| 9/10/1981  | 15.9   |
| 9/11/1981  | 15.76  |
| 9/14/1981  | 15.27  |
| 9/15/1981  | 15.16  |
| 9/16/1981  | 15.1   |
| 9/17/1981  | 15.18  |
| 9/18/1981  | 15.07  |
| 9/21/1981  | 14.98  |
| 9/22/1981  | 15.51  |
| 9/23/1981  | 14.94  |
| 9/24/1981  | 14.94  |
| 9/25/1981  | 15.02  |
| 9/28/1981  | 14.82  |

```
url = f"https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}"
```

```
df = pd.read_csv(url)
```

- maps "3m" → "DGS3MO"
- sets end = 2026-02-19
- sets start = 2026-01-19 (30 days back)
- downloads the CSV from FRED
- filters to that date range

- takes the newest yield in that range (say 5.12)
- returns 0.0512

### **src/Binomial\_model.py (CRR Binomial pricing)**

Implements the Cox-Ross-Rubinstein Binomial

- builds up/down factors  $u$ ,  $d$ ,  $u_d$ ,  $d_u$  from volatility and  $dt = T/N$ 
  - Recombines (CRR trick) -  $N+1$  distinct prices not  $2^N$
- computes risk-neutral probability  $p$ 
  - Grow at risk-free rate not the real expected return
- constructs a stock-price binomial tree (Lattice)
- prices the option via **backward induction**
  - Discount from end to  $V[0]$  = option price today

Output: Option price

### **American:**

Exercise now during the backward pass

Inside the loop → compute the immediate exercise value and take  $\max(\text{hold\_value}, \text{exercise\_value})$  at each node.

Hold Value: “If I don’t exercise now and instead wait one step, what is the discounted expected value?”

Call:  $\max(S-K, 0)$

Put:  $\max(K-S, 0)$

## Building stock and option value lattices (CRR)

- Heatmap-style lattice visualization
- Node/edge "tree look" visualization with values encoded by color
- Maturity payoff comparison: intrinsic payoff vs option value at maturity

